Scapy Intro Mini Notes

Whats the point of Scapy? Scapy is a powerful Python library used for network packet manipulation and analysis, enabling users to create, send, capture, and analyze network traffic!

To use Scapy in your IDE, you can import it by typing

```
from scapy.all import *
```

Scapy Basics (Creating Packets)

Scapy `send`

To send a ICMP Packet

```
ping = IP(dst='0.0.0.0') / ICMP()
send(ping)
```

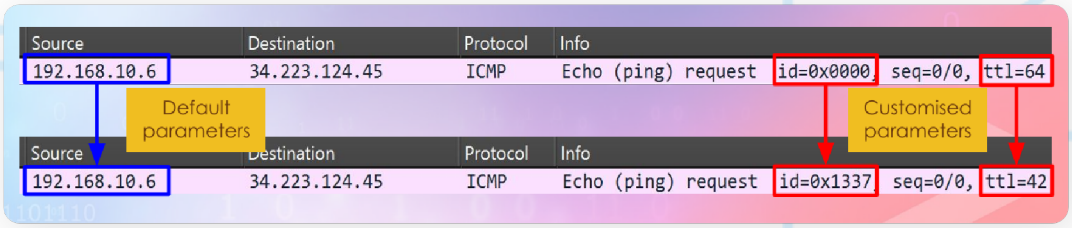
To send a DNS Query Packet

```
dnsquery = IP(dst='8.8.8.8') / UDP(dport=53) /
DNS(rd=1, qd=DNSQR(qname='www.yourdomain.com'))
send(dnsquery)
```

You can also add more parameters (with reference to sending ICMP packet), in this case the additional stuff added are `ttl` and `id`

```
ping = IP(dst='0.0.0.0', ttl=42) / ICMP(id=0x1337)
send(ping)
```

After changing parameters, it would be different from the original packet itself.



Viewing Scapy Packets

Scapy `str`

View a summary of the packet created

```
ping = IP(dst='0.0.0.0', ttl=42) / ICMP(id=0x1337)
str(ping)
```

Scapy `packet.show()`

View all the parameters that is set in the packet

```
ping = IP(dst='0.0.0.0', ttl=42) / ICMP(id=0x1337)
ping.show()
```

Scapy `raw`

View raw data of packet created

```
ping = IP(dst='0.0.0.0', ttl=42) / ICMP(id=0x1337)
raw(ping)
```

Scapy `hexdump`

View the data (but in hexadecimal) of packet created

```
ping = IP(dst='0.0.0.0', ttl=42) / ICMP(id=0x1337)
hexdump(ping)
```

Scapy Subnetting

CIDR

Using CIDR to send ICMP packets to a range of IP addresses

```
pings = IP(dst='192.168.0.50/30') / ICMP
```

Issue the name of the packet as a command to inspect the packet created

```
pings
<IP frag=0 proto=icmp dst=192.168.0.50/30> <ICMP>
```

Alternatively you can use `list` to see the range of IPs

```
list(pings)
<IP frag=0 proto=icmp dst=192.168.0.50> <ICMP>
<IP frag=0 proto=icmp dst=192.168.0.51> <ICMP>
<IP frag=0 proto=icmp dst=192.168.0.52> <ICMP>
```

```
send(pings)
Sent 4 packets.
```

Custom Parameter Range

You can issue custom Auto Adjustment Logic Range

```
pings = IP(dst='192.168.0.50', ttl=(30,32)) / ICMP
```

Issue the name of the packet as a command to inspect the packet created

```
pings
<IP frag=0 ttl=(30,32) proto=icmp dst=192.168.0.50>
<ICMP>
```

Alternatively you can use `list` to see the range of IPs

```
list(pings)
<IP frag=0 ttl=30 proto=icmp dst=192.168.0.50>
<ICMP>
<IP frag=0 ttl=31 proto=icmp dst=192.168.0.51>
<ICMP>
<IP frag=0 ttl=32 proto=icmp dst=192.168.0.52>
<ICMP>
```

```
send(pings)
Sent 3 packets.
```

Sending & Receiving

`sr1` function to send and receive ONE packet

```
# Create "ping" packet first / Or create any packet
sr1(ping, verbose=0)
```

`sr` function to send and receive MULTIPLE packets

```
# Create "ping" packet first / Or create any packet
sr(ping, verbose=0)
```

`sendp()`, `srp()`, `srp1()` Sending Layer 2 Packets

Compared to `send` which interacts with layer 3 and handles the routing and lower layers, `sendp()`, `srp()`, `srp1()` requires the user to specify the interface and handle the link layer protocol.

```
broadcast_ping = Ether(dst='ff:ff:ff:ff:ff:ff') / IP
(dst='192.168.10.255') / ICMP()
sendp(broadcast_ping)
```

Scapy Sniffing

`sniff()` function

Example of a sniff fucntion for TCP traffic @ port 80

```
sniff(filter='tcp and port 80', count=3)
<Sniffed: TCP:3 UDP:0 ICMP:0 Other:0>
```

Example of sniff for ICMP traffic at IP 8.8.8.8

```
sniff(filter='icmp and host 8.8.8.8', count=2)
<Sniffed: TCP:0 UDP:0 ICMP:2 Other:0>
```

`prn` Additional Parameters for `sniff()`

`prn` used to tell Scapy what to do with each packet it captures.

```
# Function to Print the Packets
def log_packet(pkt):
    print(pkt)

# Call for log_packet in the prn arguement
sniff(filter='icmp', prn=log_packet)
Ether / IP / ICMP 192.168.106.99 > 8.8.8.8 echo-
request 0 /Raw
Ether / IP / ICMP 8.8.8.8 > 192.168.106.99 echo-
reply 0 /Raw
Ether / IP / ICMP 192.168.106.99 > 8.8.8.8 echo-
request 0 /Raw
Ether / IP / ICMP 8.8.8.8 > 192.168.106.99 echo-
reply 0 /Raw
```

```
# Function to filter TCP packets, Destination IP and
Destination Protocol
def log_outgoing(pkt):
    if pkt[TCP].flags == 'S':
        dst_ip = pkt[IP].dst
        dst_prot = pkt[TCP].dport
        print("Detected outgoing connection to {}:
{}".format(dst_ip, dst_prot))

# Call for log_outgoing in the prn arguement
sniff(filter='tcp', prn=log_outgoing)
Detected outgoing connection to 34.223.124.45:80
Detected outgoing connection to 34.223.124.45:80
Detected outgoing connection to 45.87.221.154:6881
Detected outgoing connection to 51.81.201.183:443
```

Writing & Reading Scapy Packets

`wrpcap` Writing from Scapy

```
# Writes capstured file into
pings = IP(dst='208.80.163.232', ttl = (30,32) /
ICMP()
wrpcap('capture.pcap', pings)
```

`rdpcap` Reading from PCAP

```
# Assuming that capture.pcap exists
rdpcap ('capture.pcap')
```

`_[0]` Printing First Packet

`_[0]` is used to access the first packet in a list of packets that have been captured or generated

```
# Assuming that capture.pcap exists
rdpcap ('capture.pcap')

_[0]
<IP version=4 ihl=5 tos=0x0 len=44 id=1 flags=DF
frag=0
ttl=50 proto=tcp chksum=0x13c5 src=192.168.10.6
dst=208.80.153.232 |<ICMP type=echo-reply code=0
chksum=0x0 id=0x0 seq=0x0 |>>
```